

Presentation_Group8 逐字稿

类 Rust 编译前端与四元式中间代码生成

主讲顺序：冉子易 → 高胜寒 → 李堂辉 主讲约 6 分 50 秒，预留约 1 分钟问答

冉子易：开场与大作业 1，第 1–8 页，约 2 分 25 秒

第 1 页，封面，约 10 秒

各位老师好，我们是第 8 组。汇报题目是“编译原理大作业 1 和 2”，内容是类 Rust 语言的前端分析和四元式中间代码生成。封面姓名按学号顺序排列。

翻到第 2 页，主线，约 20 秒

对标老师的两个要求 PPTX，这两份作业可以串成一条前端流程。大作业 1 负责 Token 和 AST；大作业 2 继续消费 AST，做语义检查，并生成四元式 IR。

翻到第 3 页，对标要求，约 20 秒

这一页对应老师两个要求 PPTX。必做语法规则覆盖到 5.1；扩展结构包括引用、数组、元组、表达式块和三类循环；大作业 2 重点完成语义诊断和四元式 IR 生成。

翻到第 4 页，AST 接口，约 20 秒

这一页是两份作业衔接的核心。作业 1 输出的 AST 保留了类型、左值、控制流和表达式形态。作业 2 直接按这些节点做类型检查、循环检查，并继续生成 IR。

翻到第 5 页，扩展语法结构，约 20 秒

老师 PPTX 中 0.1 到 5.1 是最低要求。我们比较值得讲的是后续扩展规则：引用、可变引用、数组、元组、表达式块、if 表达式和三类循环，都被保留成后续可以分析的结构。

翻到第 6 页，Parser 层次，约 20 秒

Parser 使用手写递归下降。顶层解析函数、代码块和语句；表达式层再处理区间、比较、加减、乘除、调用、索引和引用。这样的层次方便维护，也方便定位错误。

翻到第 7 页，AST 设计，约 20 秒

AST 保留后续阶段需要的信息。Type 里有 I32、引用、可变引用、数组和元组；Expr 里有二元运算、引用、索引、表达式块和 if 表达式。

翻到第 8 页，作业 1 产出，约 20 秒

作业 1 的产出是可继续编译的结构。Token 提供类别和位置，AST 提供语句、表达式和类型结构。下面由高胜寒介绍语义分析。

高胜寒：语义分析，第 9–12 页，约 1 分 30 秒

翻到第 9 页，语义状态，约 20 秒

第二份作业的特色是带状态遍历 AST。Compiler 维护 scopes、current_return_type、loop_stack，以及临时变量和标签计数器。语义是否合法，IR 怎么落地，都由这些状态一起决定。

翻到第 10 页，符号表栈，约 20 秒

符号表中的变量记录可变性、静态类型和 IR 参数。作用域用 Vec<HashMap<...>> 表示，进入块时压栈，退出块时弹栈，查找变量时从内层向外层查。

翻到第 11 页，检查点，约 25 秒

这一页对应老师要求 PPTX 里容易被追问的语义点：类型要对应，不可变变量不能二次赋值，break、continue 必须在循环内。实现上分别落到 Type 枚举、符号表里的 mutable 字段和 loop_stack。

翻到第 12 页，错误汇总，约 25 秒

错误会集中收集到 errors: Vec<String>，最后统一输出。因此同一段程序可以同时报告不可变变量赋值、循环外 break 和返回类型不一致。下面由李堂辉介绍 IR 生成。

李堂辉：IR 生成、验证与收束，第 13–19 页，约 2 分 45 秒**翻到第 13 页，IR 结构，约 20 秒**

大作业 2 要求实现中间代码生成，并建议用四元式。我们的 IR 每条指令包含 op、arg1、arg2 和 result；Arg 可以是空值、整数、变量、临时变量或标签。

翻到第 14 页，表达式翻译，约 25 秒

表达式翻译时，compile_expression 返回 (Type, Arg)。例如 $a + b * 2$ 先生成 $b * 2$ 的 t0，再生成 $a + t0$ 的 t1，最后把 t1 赋值给 c。

翻到第 15 页，分支翻译，约 20 秒

分支结构用条件跳转和标签表示。先计算条件，再用 JUMP_IF_FALSE 跳到 else 标签；then 分支结束后跳到结束标签。

翻到第 16 页，循环翻译，约 20 秒

循环结构依赖入口标签、出口标签和循环栈。continue 跳入口，break 跳出口。编译循环体时把两个标签压入 loop_stack。

翻到第 17 页，扩展结构落点，约 30 秒

这一页讲扩展结构在第二份作业里的真实落点。引用生成 REF，解引用生成 Deref，数组读取生成 LOAD_ARRAY。元组下标会根据元组类型推导元素类型；函数调用先生成参数 ARG，再生成 CALL。函数签名检查和数组写入目前作为后续扩展说明。

翻到第 18 页，测试覆盖，约 25 秒

测试也按老师规则组织。`test_all.rs.txt` 覆盖主要语法结构，`test_errors.rs.txt` 覆盖词法和语法错误，`test_semantic_errors.rs.txt` 覆盖未声明、类型不匹配、不可变赋值和非法控制流等语义错误。

翻到第 19 页，总结，约 30 秒

总结一下，两份作业完成了从类 Rust 源程序到四元式 IR 的前端流程。作业 1 把语法结构化，作业 2 带状态遍历 AST，检查作用域、类型、可变性、返回类型和循环上下文。最终 IR 用临时变量和标签线性化表达式和控制流。我们的汇报到这里结束，谢谢老师，欢迎提问。

备用问答提示，不计入主讲时间

如果老师问两份作业区别

大作业 1 的输入是源代码字符串，核心状态是 Token 指针和解析函数栈，输出是 Token 和 AST。大作业 2 的输入是 AST，核心状态是符号表栈、循环栈、返回类型和计数器，输出是语义错误或四元式 IR。

如果老师问实现边界

当前重点是前端流程和四元式表达，没有继续做目标代码生成。函数调用已经生成 ARG/CALL，但函数签名检查还可以继续完善。数组读取有 LOAD_ARRAY，数组写入接口存在，后续可以扩展更完整的地址计算和 STORE_ARRAY。

如果老师问实现特色

第一，两份作业用 AST 接成一条流程；第二，引用、数组、元组、表达式块和循环结构都进入 AST；第三，语义分析带着 scopes、current_return_type 和 loop_stack 运行；第四，IR 用临时变量和标签把表达式、分支和循环线性化。这些点都能在 ast.rs、compiler.rs 和 ir.rs 中对应到代码。

如果老师问和要求 PPTX 的对应关系

必做语法规则覆盖到老师要求的 5.1；扩展规则中，引用、数组、元组、表达式块、循环控制进入 AST，并在语义或 IR 阶段有对应处理。大作业 2 对应语义诊断和四元式生成，重点实现了类型一致、可变性、返回类型、循环控制和标签化 IR。